Homework 2

Operating Systems • Inha University in Tashkent • Fall 2026

Assigned	10.10.2026	**Instructor**	Yusupov Jalolliddin
Due	24.10.2026, 23:55 (UTC+5)	**Weight**	100 points / 40 = (2.5%)
Submit	One ZIP with source code	**Work**	Individual

What you submit

```
SOC3010_HW2_<StudentID>_<Surname>.zip
├── report.pdf                          ← answers, screenshots, photos
└── src/
    ├── ……
```

The report is **one PDF**. Write it in Word or LibreOffice (**using the template provided**) and export to PDF. Number your answers exactly as this handout numbers the questions — 1.1, 2.3, 5.4 — so we can find them.

Given files (eClass, `hw2-starter.zip`): You never edit `shared_mem.h` or `net.h`; you include them. Work on your **Ubuntu VM**: macOS does not implement `sem_init`.

Compiling. `gcc -o name name.c`, then `./name`. Anything that uses a semaphore also needs `-pthread` at the end: `gcc -o counter_sem2 counter_sem2.c -pthread`. Keep `net.h` and `shared_mem.h` in the same folder as your `.c` files.

The rules for screenshots

Most of the marks are for screenshots. A screenshot only counts if **all three** of these are true:

1. It is a picture of **your own terminal on your own VM**. The shell prompt must be visible — the part that looks like `student@ubuntu:~$`. Crop the window if you like, but never crop the prompt away.
2. The **command you typed is visible**, not only its output. We need to see what produced the result.
3. It is **readable**. A photograph of your monitor taken with a phone is not readable. Use your VM's screenshot key (`PrtSc`) or the Screenshot application.

A screenshot with no prompt in it is worth zero marks. It could have come from anyone.

Two terminals. Many tasks need a server in one terminal and clients in another. Put both windows in one screenshot, or two screenshots side by side, and label them "server" and "client".

The rules for handwriting

Parts 2, 5 and 6 are done **on paper, by hand**. Write your name and student ID at the top of every sheet, photograph the sheets, and put the photographs in the report. Typed descriptor tables, typed timing tables and typed trace tables score zero. The midterm will ask for exactly these skills.

Part 1 — Try it on your machine

25 points

Nothing to invent here. You run programs from the lectures, downloaded from eClass, and report what your own machine does. For every item: **run it, screenshot it, and explain in one to three sentences what the OS did.**

1.1 — Read until 0

Run `pipe_eof`. The child writes three lines into a pipe and exits; the parent reads until `read` returns 0.

Hand in: a screenshot, and two sentences: what does `read` returning 0 mean, and which **two** `close`s (one in each process) made it possible?

> **The line `setvbuf(stdout, NULL, _IONBF, 0);` — read this once, then use it everywhere**
>
> When your program calls `printf`, the text does **not** go to the screen immediately. It goes into a **buffer**: a small waiting area in your process's memory. The buffer is emptied ("flushed") later — on a terminal at every newline, otherwise when it is full or when the program exits. Buffering exists for speed: one trip to the OS for many characters instead of one trip per character.
>
> Normally you never notice. With `fork()` you do. `fork()` copies the **whole** process — and the buffer is part of it. If some text is still waiting in the buffer when `fork()` runs, the parent **and** the child both have a copy of it, and both print it when they exit. You see the same line twice, although `printf` ran only once. (Try 1.3: `twice.c` shows exactly this.)
>
> `setvbuf` changes how the buffer works. Its four arguments: `stdout` (the stream `printf` writes to), `NULL` (do not give it a buffer of our own), `_IONBF` (**no buffering**: every `printf` goes straight to the OS), `0` (buffer size: none). After this line, every `printf` is on the screen *before* the next line of your program runs, so there is nothing left to copy.
>
> **Rule for this course:** make it the first line of `main` in every program that calls `fork()`. It costs a little speed and saves you an hour of confusion.

1.2 — One forgotten close

In `pipe_eof.c`, put the line marked `THE line of Part 1.2` (the parent's `close(p[1])`) inside `/* … */`, compile and run again. The program prints the three lines and then nothing, forever. Stop it with `Ctrl-C`.

Hand in: a screenshot of the hang, and two sentences: which process is asleep, in which call, and **whose** write end is it waiting for?

1.3 — The line that prints twice

Run `twice`. It prints `hello world` twice, although `printf("hello ")` runs once. Now remove the `/*` and `*/` around the `setvbuf` line, compile and run again.

Hand in: both screenshots, and two sentences: where was `"hello "` when `fork` ran?

1.4 — A pipe with a name

```
# terminal 1                        # terminal 2
$ mkfifo /tmp/hw2fifo                 $ echo "hello from bash" > /tmp/hw2fifo
$ ls -l /tmp/hw2fifo
$ cat /tmp/hw2fifo
```

Hand in: a two-terminal screenshot, and answers: what is the first letter of the `ls -l` line and the file's size, and why?

1.5 — Who waits at the door

`writer` and `reader`, three runs:
(a) `rm -f /tmp/demo_fifo`, then `./reader` alone;
(b) `./writer` in terminal 1, wait five seconds, then `./reader` in terminal 2;
(c) `./reader` first, then `./writer`.

Hand in: one screenshot per run.

For (a): why the error? For (b) and (c): which program waited, in which call, and what woke it?

1.6 — The lost update

Run `counter_race2` three times and `counter_sem2` three times.
Then run each once more as `time ./counter_race2` and `time ./counter_sem2`.

`time` in front of any command runs it and then prints how long it took; read the `real` line.

Hand in: one screenshot of all the runs, and three sentences: why the first three numbers differ from each other; why the last three are exactly 2000000; which program is slower and why.

1.7 — Taking turns

Run `pingpong` and `order_sem`.

Hand in: one screenshot of both, and answers: in `pingpong`, which semaphore starts at 1 and which at 0, and what would the program print if both started at 0? In `order_sem`, the parent sleeps before it prints `A`. If you change `sleep(1)` to `sleep(5)`, can `B` come first? Why not?

1.8 — Your first server

Terminal 1: `./echo_server`.
Terminal 2: `./client localhost hello`, then `./client localhost "operating systems"`.

Hand in: a two-terminal screenshot, and one sentence: which line appears in the server's terminal when a client hangs up, and what did the server's `read` return at that moment?

1.9 — Two errors, on purpose

(a) Stop the server with `Ctrl-C` and run `./client localhost hello`.
(b) Start `./echo_server` in terminal 1 and start it **again** in terminal 2.

Hand in: a screenshot of each error, and one sentence each: what did the OS tell you, and why?

1.10 — Leave the machine (optional)

Find your machine's address with `hostname -I`.
Option 1: on the same Wi-Fi, a classmate runs `./client <your address> hello` against your `echo_server`.
Option 2 (alone, or if your VM's network is NAT and nobody can reach it):
run the client on your own VM with your address instead of `localhost`.

Hand in: a screenshot showing `hostname -I`, the server's terminal and the client's command with the address. Say which option you used.

1.11 — One operator or many

Terminal 1: `./server_one`.
Terminal 2, all on one line:
`./timed_client localhost A & ./timed_client localhost B & ./timed_client localhost C &`
Stop the server with `Ctrl-C` and repeat with `./server_fork`.

Hand in: both screenshots, and two sentences: explain the two sets of times (2, 4, 6 and 2, 2, 2). While the first client was being served by `server_one`, where exactly were the other two?

Part 2 — Pipes and descriptors, by hand

10 points

Handwritten on paper and photographed. No computer.

2.1 — Count the ends

```
int p[2];
pipe(p);
if (fork() == 0) {                  /* child */
    dup2(p[1], 1);
    close(p[0]);
    close(p[1]);
    execlp("ls", "ls", NULL);
}
close(p[1]);                        /* parent */
while ((n = read(p[0], buf, sizeof buf)) > 0)
    write(1, buf, n);
wait(NULL);
```

Draw the descriptor tables (fd 0 to 4) of the parent and of the child at three moments:
(1) right after `fork`; **(2)** after the child's `dup2` and its two `close`s; **(3)** after the parent's `close(p[1])`.
Write in each box what the descriptor refers to: terminal, read end, write end, or free.

Hand in: the photograph, and two answers:
(a) at moment 3, how many descriptors in the whole system refer to the **write end**, and who holds them?
(b) The parent's `read` returns 0 only after two things have happened. Which two?

2.2 — Finish, sleep forever, or killed?

For each program only say **finishes**, **sleeps forever** or **killed by a signal**;
name the process and the call where it stops, and what it prints first.

```
/* (a) */
int p[2];
pipe(p);
if (fork() == 0) {
    close(p[0]);
    write(p[1], "one\ntwo\nthree\n", 14);
    exit(0);
}
while ((n = read(p[0], buf, 100)) > 0)
    write(1, buf, n);
printf("parent: done\n");
wait(NULL);

/* (b) */
int down[2], up[2];
pipe(down);
pipe(up);
if (fork() == 0) {
    close(down[1]); close(up[0]);
    read(down[0], buf, sizeof buf);
    write(up[1], "hi parent", 10);
    exit(0);
}
close(down[0]); close(up[1]);
read(up[0], buf, sizeof buf);
write(down[1], "hi child", 9);
wait(NULL);


/* (c) — nobody else opens this FIFO */
mkfifo("/tmp/hw2_lonely", 0666);
printf("opening...\n");
int fd = open("/tmp/hw2_lonely", O_RDONLY);
printf("opened, fd = %d\n", fd);
close(fd);
return 0;
```

Hand in: the photograph or just text, one line per program.

Part 3 — Pipes and FIFOs in C

15 points

`pipeline3.c`

Write the shell line `ls /etc | grep conf | wc -l` in C:
two pipes, three children, and a parent that behaves like the shell. Usage:

```
$ ls /etc | grep conf | wc -l
14
$ ./pipeline3
14
3 children finished
```

`grep conf` reads lines from standard input and prints only those that contain `conf`.
`wc -l` counts the lines it reads and prints the number. Your number may differ from 14; it must be the same as the shell line's on **your** machine.

#	Requirement
1	Both pipes are created **before the first** `fork`.
2	Child 1 sends its standard output into pipe 1 and becomes `ls /etc`; child 2 reads standard input from pipe 1, sends standard output into pipe 2 and becomes `grep conf`; child 3 reads standard input from pipe 2 and becomes `wc -l` — each with `dup2`, then `execlp`.
3	**The Sacred Rule:** every child closes all four pipe descriptors after its `dup2`s; the parent closes all four.
4	The parent waits for all three children, then prints `3 children finished`.
5	Screenshot: the shell line and `./pipeline3` printing the same number.
6	**Break it on purpose:** put the parent's `close` of pipe 2's write end inside `/* … */`, run, screenshot the hang, stop it with `Ctrl-C`. Two sentences: which process sleeps in which call, and whose descriptor is it waiting for? Restore the line.

Hand in: `pipeline3.c` in `src/`, the screenshots of rows 5 and 6, and the two sentences of row 6.

Part 4 — Sockets in C

25 points

4.1 — `greet_server.c` and `greet_client.c`

A call centre in which one call can carry **many messages**. The client dials once and sends one line per name; the server gives every client its own operator (a process per client), and the operator answers every line until the client hangs up. Usage, in three terminals:

```
# terminal 1
$ ./greet_server
receptionist 510: waiting on port 5000
operator 516: client left after 2 messages
operator 515: client left after 3 messages


# terminal 2
$ ./greet_client localhost Ali Vali Sami
client: hello Ali, I am operator 515 (message 1)
client: hello Vali, I am operator 515 (message 2)
client: hello Sami, I am operator 515 (message 3)



# terminal 3, at the same time
$ ./greet_client localhost Lola Aziz
client: hello Lola, I am operator 516 (message 1)
client: hello Aziz, I am operator 516 (message 2)
```

#	Requirement
1	**Client:** `client_connect` once; for every name from `argv[2]` on, write the name and `"\n"`, `read_line` the answer and print it, then `sleep(1)`; `close` at the end.
2	**Server:** `server_listen(5000)` once; then forever: `server_accept`, `fork`.
3	The child closes `L`; the parent closes `c` right after the `fork`; the child `exit`s after serving.
4	The child loops on `read_line` until it returns 0 and answers every line with `hello <name>, I am operator <its PID> (message <k>)` and `"\n"`.
5	When the client hangs up, the operator prints `operator <PID>: client left after <k> messages`; the receptionist prints its PID once at the start.
6	No zombies: Homework 1's `SIGCHLD` handler, installed with `sigaction` and `SA_RESTART`, with the `waitpid(-1, NULL, WNOHANG)` loop — as in `server_fork.c`.
7	Screenshots: the three terminals with two clients running at the same time; then `ps -o pid,ppid,stat,comm --ppid <receptionist PID>` after both finished, showing no `Z`.

Hand in: both files in `src/`, the screenshots of row 7, and two sentences: why does each client see a different operator number, but the same number for all of its own messages?

4.2 — Timing the designs

Copy `server_pool.c` to `server_pool3.c` and change `POOL` to 3.
Before you run anything, fill the "predicted" column for five clients that each need 2 seconds.
Then, for each of `server_one`, `server_fork`, `server_pool` and `server_pool3`:
start the server in terminal 1, run the line below in terminal 2, screenshot, stop the server.

```
$ for n in A B C D E; do ./timed_client localhost $n & done
```

Server	Predicted reply times (s)	Measured reply times (s)	Server processes at the busiest moment
`server_one`			
`server_fork`			
`server_pool` (2)			
`server_pool3`			

Hand in: `server_pool3.c` in `src/`, the filled table (this one table may be typed), the four screenshots, and two sentences: which client got which time changed from run to run — did the **set** of times change?
Why? *(Points: predicted column 2, screenshots 2, processes column 1, sentences 1.)*

Part 5 — Sockets and servers, by hand

10 points

Handwritten on paper and photographed. `serve(C)` is Slide 11's: `name = read(C)`, work, `write(C, "done " + name)`.

5.1 — Six clients, three seconds each

Clients A, B, C, D, E, F connect at the same moment, in that order; every request needs **3 seconds** of work.
Copy the table and fill in when each client gets its reply,
when everything is done, and how many server processes exist at the busiest moment.

Server	A	B	C	D	E	F	done	processes
Design 1: one at a time								
Design 2: a process per client								
Design 3: a pool of 2								
Design 3: a pool of 4								

Hand in: the photograph of the filled table.

5.2 — Three more broken servers

Each is Slide 11's process-per-client server with one change. Three clients A, B, C (2 seconds of work each) connect at the same moment. For each server: what do the clients see (their reply times, or no reply)?
Is it broken at all? If it is, write the one-line fix.

```
Server P:
loop forever:
    C = accept(L)
    if fork() == 0:
        close(L)
        serve(C)
        close(C)
        exit
    close(C)
    wait()        /* new */
Server Q:
loop forever:
    C = accept(L)
    if fork() == 0:
        close(L)
        serve(C)
        exit      /* no close(C) */
    close(C)
Server R:
loop forever:
    C = accept(L)
    if fork() == 0:
        close(L)
        serve(C)
        close(C)
        exit
    close(L)      /* L instead of C */
```

Hand in: the photograph, three to four lines per server.

5.3 — Who sleeps where?

```
Server:   (the echo server: one at a time)
L = listen(5000)
loop forever:
    C = accept(L)
    while (n = read(C)) > 0:
        write(C, what was read)
    close(C)
Client A:   (starts at t = 0)
C = connect(server, 5000)
sleep 60        /* lunch */
close(C)

Client B:   (starts at t = 1)
C = connect(server, 5000)
write(C, "hi\n")
reply = read(C)
print(reply)
```

Hand in: the photograph with
(a) at t = 10 s, for each of the three processes: asleep or not, in which call, and what will wake it;
(b) at what time B prints `hi`;
(c) what changes if the server is a process per client?

5.4 — A stream of bytes

```
Client:
C = connect(server, 5000)
write(C, "AB")
write(C, "CD\n")
write(C, "EF\n")
close(C)

Server (a):
C = accept(L)
n = read(C, buf, 100)
print(buf)

Server (b):
C = accept(L)
print(read_line(C))
print(read_line(C))
```

Hand in: the photograph.
(a) For each, possible or impossible, with a reason: `AB` · `ABCD\nEF\n` · `A` · `CD\n` · `ABC`.
(b) What does server (b) print — and can it ever print anything else?

Part 6 — Semaphores, by hand

15 points

Handwritten on paper and photographed. Trace tables in Slide 9's form: `# · operation · S · sleeping · inside`.
The golf hole: the value is the number of flags in the hole;
`wait` takes a flag or sleeps beside the hole; `post` puts a flag back or hands it to a sleeper.

6.1 — Trace the value

S starts at **2**. Fill one row per step:

#	operation	S	sleeping	inside	#	operation	S	sleeping	inside
1	P1: wait(S)				6	P4: post(S)			
2	P2: wait(S)				7	P5: wait(S)			
3	P3: wait(S)				8	P2: post(S)			
4	P4: wait(S)				9	P3: post(S)			
5	P1: post(S)				10	P5: post(S)			

Hand in: the photograph, and one sentence: at step 5 you could not know who got the flag.
Which later step told you?

6.2 — Which outputs are possible?

```
(a)   s = 0, t = 0
Process P1:             Process P2:             Process P3:
    print("A")              wait(s)                 wait(t)
    post(s)                 print("C")              print("D")
    print("B")
    post(t)

(b)   one lock L, free at the start
Process P1:             Process P2:             Process P3:
    lock(L)                 lock(L)                 lock(L)
    print("A")              print("C")              print("E")
    print("B")              print("D")              print("F")
    unlock(L)               unlock(L)               unlock(L)
```

Hand in: the photograph.
(a) **[3]** List every output, with a one-line reason each, and answer: can the output be `ADBC`?
(b) **[1]** How many different outputs are possible? List them.

6.3 — Fill the blanks

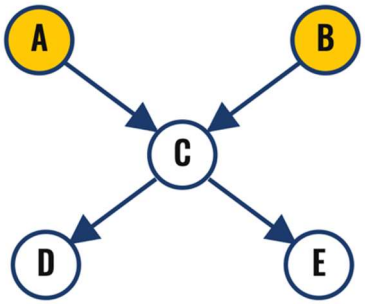
The output must always be **A B C D**, whatever the scheduler does.
Fill the blanks with `wait(…)` or `post(…)`, one semaphore per arrow, and give every start value.
Some blanks stay empty.

```
Process P1:          Process P2:          Process P3:

    ________             ________             ________
    print("A")           print("B")           print("C")
    ________             ________             ________
    ________
    print("D")
```

Hand in: the photograph, and one sentence: a classmate uses **one** semaphore for all three arrows.
Give one output his program can produce that yours cannot.

6.4 — From a drawing to a program



Five processes PA … PE print one letter each.
An arrow X → Y means X must be printed before Y.

Hand in: the photograph with
(a) how many semaphores (minimum) and their start values;
(b) the five processes in pseudo-code;
(c) every output that is possible.

6.5 — Two keys

```
S = 1, T = 1            /* two semaphores used as locks */

Process P1:              Process P2:
    wait(S)                  wait(T)
    wait(T)                  wait(S)
    print("X")               print("Y")
    post(T)                  post(S)
    post(S)                  post(T)
```

Deadlock (Slide 6 used the word for two pipes): a group of processes in which every one is asleep, waiting for a flag that only another sleeper in the group could post. Nobody ever wakes.

Hand in: the photograph with
(a) a trace table of four steps after which both processes sleep forever;
(b) a change to **one** process that makes this impossible, and why it works.